

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0716

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Network Throughput and Packet Loss

Problem Understanding

Inputs:

- Total data received = 8000 bits
- Time = 4 seconds
- Packets sent = 200
- Packets received = 180

Outputs:

- Throughput in bits per second
- Packet loss percentage
- Network classification

Constraints:

- Time must not be zero.
- Total packets sent must not be zero.

Formula

Throughput:

Throughput = Total Data Received / Time

Packet Loss:

Lost Packets = Packets Sent - Packets Received

Packet Loss % = (Lost Packets / Packets Sent) × 100

Pseudocode

BEGIN

totalDataReceived ← 8000

time ← 4

packetsSent ← 200

packetsReceived ← 180

IF time = 0 THEN

 DISPLAY "Error: Time cannot be zero"

 STOP

END IF

IF packetsSent = 0 THEN

 DISPLAY "Error: Packets sent cannot be zero"

 STOP

END IF

throughput $\leftarrow$ totalDataReceived / time

lostPackets $\leftarrow$ packetsSent - packetsReceived

IF lostPackets < 0 THEN

 DISPLAY "Error: Received packets cannot exceed sent packets"

 STOP

END IF

packetLoss $\leftarrow$ (lostPackets / packetsSent) $\times$ 100

IF throughput > 1500 AND packetLoss < 10 THEN

 status $\leftarrow$ "Efficient"

ELSE

 status $\leftarrow$ "Inefficient"

END IF

DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"

DISPLAY "Total Data Received: ", totalDataReceived, " bits"

DISPLAY "Time: ", time, " seconds"

DISPLAY "Packets Sent: ", packetsSent

DISPLAY "Packets Received: ", packetsReceived

DISPLAY "Lost Packets: ", lostPackets

DISPLAY "Throughput: ", throughput, " bps"

DISPLAY "Packet Loss: ", packetLoss, "%"

DISPLAY "Network Status: ", status

END

Calculation

Throughput:

$$8000 / 4 = 2000 \text{ bps}$$

Lost packets:

$$200 - 180 = 20 \text{ packets}$$

Packet loss:

$$(20 / 200) \times 100 = 10\%$$

Since:

- Throughput = **2000 bps** > **1500 bps** ✓
- Packet loss = **10%** is **NOT less than 10%** ✗

Therefore:

Network Status = Inefficient

Rubric Coverage

- **Problem Understanding:** All inputs, formulas, constraints, and outputs identified.
 - **Algorithm Design:** Calculates throughput and packet loss correctly.
 - **Control Structures:** Uses IF conditions for validation and classification.
 - **Pseudocode Structure:** Clear sequential structure.
 - **Completeness:** Handles zero values and invalid packet counts.
-

2. Bandwidth Utilization Monitoring

Problem Understanding

The organization has multiple network links. The algorithm must:

1. Monitor every link.
2. Collect bandwidth usage periodically.
3. Store values in an array.
4. Check utilization.
5. Detect utilization above **80%**.
6. Recommend switching traffic to a backup link.
7. Continue monitoring repeatedly.

Pseudocode

BEGIN

```
links ← ["Link1", "Link2", "Link3", "Link4"]
utilization[4]
```

WHILE TRUE DO

```
    DISPLAY "----- BANDWIDTH MONITORING -----"
```

```
    FOR i ← 0 TO 3 DO
```

```
        utilization[i] ← GET_BANDWIDTH_UTILIZATION(links[i])
```

```
        DISPLAY links[i], ": ", utilization[i], "%"
```

```
        IF utilization[i] > 80 THEN
```

```
            DISPLAY "Warning: ", links[i], " is overloaded"
```

```
            DISPLAY "Recommendation: Switch traffic to backup link"
```

```
            SWITCH_TO_BACKUP(links[i])
```

```
        ELSE
```

```
            DISPLAY links[i], " is operating normally"
```

```
        END IF
```

```
    END FOR
```

```

        WAIT 10 seconds

    END WHILE

END

Improved Version with Backup Availability
BEGIN

    links ← ["Link1", "Link2", "Link3", "Link4"]
    utilization[4]

    WHILE TRUE DO

        FOR i ← 0 TO 3 DO

            utilization[i] ← GET_BANDWIDTH_UTILIZATION(links[i])

            IF utilization[i] > 80 THEN

                DISPLAY "ALERT: ", links[i], " exceeds 80% utilization"

                IF BACKUP_LINK_AVAILABLE(links[i]) THEN
                    SWITCH_TO_BACKUP(links[i])
                    DISPLAY "Traffic switched to backup link"
                ELSE
                    DISPLAY "No backup link available"
                END IF

            ELSE
                DISPLAY "Link operating normally"
            END IF

        END FOR

        WAIT 10 seconds

    END WHILE

END

```

How It Supports Bandwidth Management

- The FOR loop checks **every network link**.
- The utilization values are stored in an array for monitoring.
- The WHILE loop provides **continuous monitoring**.
- The IF condition detects utilization above **80%**.
- Traffic is moved to a backup link when available.
- Early detection prevents a link from becoming severely congested.
- Periodic monitoring allows the network to respond to changing traffic conditions.

Rubric Coverage

Criterion	How the algorithm satisfies it
Problem Understanding	Identifies links, utilization, threshold and backup
Algorithm Design	Continuously monitors and responds to overload
Control Structures	Uses WHILE, FOR, and IF
Pseudocode Structure	Clear monitoring sequence
Completeness	Handles overloaded and normal links and backup availability

3. Dynamic Best-Path Selection

Problem Understanding

There are **six branches** connected through multiple links.

Each link has:

- Bandwidth
- Delay
- Utilization
- Link status

These values change over time.

The algorithm must:

1. Periodically collect network metrics.
2. Store metrics in arrays/lists.
3. Calculate a quality score.
4. Select the highest-scoring path.
5. Detect overloaded links.
6. Detect failed links.
7. Select an alternate path automatically.
8. Generate an administrator alert.
9. Continue monitoring.

Link Quality Formula

A simple normalized score can be:

Quality Score =
(0.5 × Normalized Bandwidth)
+ (0.3 × Normalized Delay)

+ (0.2 × Normalized Utilization)

For bandwidth, higher is better.

For delay and utilization, lower is better.

Therefore:

Normalized Bandwidth = Bandwidth / Maximum Bandwidth

Normalized Delay = 1 - (Delay / Maximum Delay)

Normalized Utilization = 1 - (Utilization / 100)

This gives a score between approximately **0 and 1**, where a higher value represents a better link.

Pseudocode

BEGIN

branches ← ["Branch1", "Branch2", "Branch3",
 "Branch4", "Branch5", "Branch6"]

WHILE TRUE DO

 DISPLAY "----- NETWORK PATH MONITORING -----"

 FOR each branch IN branches DO

 paths ← GET_AVAILABLE_PATHS(branch)

 bestScore ← -1
 bestPath ← NONE

 FOR each path IN paths DO

 bandwidth ← GET_BANDWIDTH(path)
 delay ← GET_DELAY(path)
 utilization ← GET_UTILIZATION(path)
 status ← GET_STATUS(path)

 STORE bandwidth IN bandwidthList
 STORE delay IN delayList
 STORE utilization IN utilizationList

 IF status = "FAILED" THEN
 DISPLAY "Link failure detected: ", path
 CONTINUE
 END IF

 IF utilization > 80 THEN


```
        DISPLAY "Overloaded link: ", path
        CONTINUE
    END IF

    normalizedBandwidth ← bandwidth / MAX_BANDWIDTH
    normalizedDelay ← 1 - (delay / MAX_DELAY)
    normalizedUtilization ← 1 - (utilization / 100)

    score ← (0.5 × normalizedBandwidth)
           + (0.3 × normalizedDelay)
           + (0.2 × normalizedUtilization)

    IF score > bestScore THEN
        bestScore ← score
        bestPath ← path
    END IF

END FOR

IF bestPath = NONE THEN
    DISPLAY "ALERT: No valid path for ", branch
    SEND_ALERT_TO_ADMINISTRATOR(branch)

ELSE
    SELECT_PATH(branch, bestPath)

    DISPLAY "Branch: ", branch
    DISPLAY "Selected Path: ", bestPath
    DISPLAY "Quality Score: ", bestScore
END IF

END FOR

WAIT 10 seconds

END WHILE

END
```

Example

Suppose three possible paths to a branch have the following values:

Path	Bandwidth	Delay	Utilization	Status
Path A	100 Mbps	30 ms	50%	Active
Path B	80 Mbps	15 ms	40%	Active
Path C	120 Mbps	50 ms	90%	Active

Path C has high bandwidth but **90% utilization**, so it is considered overloaded and is not selected.

The algorithm compares the remaining valid paths using their overall quality scores and selects the path with the **highest score**.

Handling Failure and Overload

If a link fails:

IF status = "FAILED"

 Ignore the link

 Search for another available path

 Send alert to administrator

END IF

If utilization exceeds 80%:

IF utilization > 80%

 Mark link as overloaded

 Do not select it as the preferred path

 Search for alternate path

 Send warning to administrator

END IF

This makes the algorithm **fault tolerant**.

How the Algorithm Improves the Network

1. Network Reliability

Failed links are detected automatically and traffic is moved to another available path.

2. Routing Efficiency

The algorithm does not simply select the shortest path. It considers:

- Bandwidth
- Delay
- Utilization

Therefore, it can select a path with better overall performance.

3. Fault Tolerance

If the selected path fails, another valid path is automatically selected.

4. Dynamic Adaptation

Network conditions are checked repeatedly. If utilization or delay changes, the quality score is recalculated and the best path can change.

5. Congestion Prevention

Links above **80% utilization** are avoided, reducing the possibility of network congestion.

6. Administrator Awareness

Alerts are generated when links fail or become overloaded, allowing administrators to take corrective action.
